JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class DoubleAccumulator

java.lang.Object
  java.lang.Number
    java.util.concurrent.atomic.DoubleAccumulator

All Implemented Interfaces:

Serializable

```
public class DoubleAccumulator
extends Number
implements Serializable
```

One or more variables that together maintain a running `double` value updated using a supplied function. When updates (method `accumulate(double)`) are contended across threads, the set of variables may grow dynamically to reduce contention. Method `get()` (or, equivalently, `doubleValue()`) returns the current value across the variables maintaining updates.

This class is usually preferable to alternatives when multiple threads update a common value that is used for purposes such as summary statistics that are frequently updated but less frequently read.

The supplied accumulator function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads. For predictable results, the accumulator function should be commutative and associative within the floating point tolerance required in usage contexts. The function is applied with an existing value (or identity) as one argument, and a given update as the other argument. For example, to maintain a running maximum value, you could supply `Double::max` along with `Double.NEGATIVE_INFINITY` as the identity. The order of accumulation within or across threads is not guaranteed. Thus, this class may not be applicable if numerical stability is required, especially when combining values of substantially different orders of magnitude.

Class `DoubleAdder` provides analogs of the functionality of this class for the common special case of maintaining sums. The call `new DoubleAdder()` is equivalent to `new DoubleAccumulator((x, y) -> x + y, 0.0)`.

This class extends `Number`, but does *not* define methods such as `equals`, `hashCode` and `compareTo` because instances are expected to be mutated, and so are not useful as collection keys.

Since:

1.8

See Also:

Serialized Form

Constructor Summary

Constructors

Constructor	Description
`DoubleAccumulator (DoubleBinaryOperator accumulatorF double identity)`	Creates a new instance using the given accumulator function and identity element.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
void	`accumulate(double x)`	Updates with the given value.
double	`doubleValue()`	Equivalent to `get()`.
float	`floatValue()`	Returns the current value as a `float` after a narrowing primitive conversion.
double	`get()`	Returns the current value.
double	`getThenReset()`	Equivalent in effect to `get()` followed by `reset()`.
int	`intValue()`	Returns the current value as an `int` after a narrowing primitive conversion.
long	`longValue()`	Returns the current value as a `long` after a narrowing primitive conversion.
void	`reset()`	

		Resets variables maintaining updates to the identity value.
String	**toString**()	Returns the String representation of the current value.

Methods declared in class Number

byteValue, shortValue

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Details

DoubleAccumulator

```
public DoubleAccumulator(DoubleBinaryOperator accumulatorFunction,
                         double identity)
```

Creates a new instance using the given accumulator function and identity element.

Parameters:

`accumulatorFunction` - a side-effect-free function of two arguments

`identity` - identity (initial value) for the accumulator function

Method Details

accumulate

```
public void accumulate(double x)
```

Updates with the given value.

Parameters:

`x` - the value

get

```
public double get()
```

Returns the current value. The returned value is *NOT* an atomic snapshot; invocation in the absence of concurrent updates returns an accurate result, but concurrent updates that occur while the value is being calculated might not be incorporated.

Returns:

the current value

reset

```
public void reset()
```

Resets variables maintaining updates to the identity value. This method may be a useful alternative to creating a new updater, but is only effective if there are no concurrent updates. Because this method is intrinsically racy, it should only be used when it is known that no threads are concurrently updating.

getThenReset

```
public double getThenReset()
```

Equivalent in effect to `get()` followed by `reset()`. This method may apply for example during quiescent points between multithreaded computations. If there are updates concurrent with this method, the returned value is *not* guaranteed to be the final value occurring before the reset.

Returns:

the value before reset

toString

```
public String toString()
```

Returns the String representation of the current value.

Overrides:

`toString` in class `Object`

Returns:

the String representation of the current value

doubleValue

```
public double doubleValue()
```

Equivalent to `get()`.

Specified by:

`doubleValue` in class `Number`

Returns:

the current value

longValue

`public long longValue()`

Returns the current value as a `long` after a narrowing primitive conversion.

Specified by:

`longValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type `long`.

intValue

`public int intValue()`

Returns the current value as an `int` after a narrowing primitive conversion.

Specified by:

`intValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type `int`.

floatValue

`public float floatValue()`

Returns the current value as a `float` after a narrowing primitive conversion.

Specified by:

`floatValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type `float`.

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.